

HDFC Bank

Two self-service journeys, opposite risk profiles — money out and information out

Both journeys move work away from a human. One moves money and cannot be undone casually; the other moves information and can. That single difference changes the design at every layer, and treating them as one class of problem is where self-service programmes go wrong.

2

journeys, one shared spine

16

use cases between them

11

request categories in one journey

1

irreversible action

6

design layers that diverge

0

shared components forked

JOURNEY SCOPE · UNASSISTED SELF-SERVICE

Digital account funding — money movement in session · loan servicing self-help — information and request handling across eleven categories

Aditya Singh · Programme and delivery ownership, digital journey portfolio · 2024

Reversibility is the axis that should decide a self-service design — not channel, not volume, and not how similar two journeys look on a roadmap

Situation

Two unassisted journeys delivered in the same year, on the same platform, for the same bank. Both remove work from an assisted channel. Both were scoped and funded as self-service builds.

On a portfolio roadmap they are the same kind of item.

Complication

One of them moves real money on an external payment rail, in a session that will expire, with a settlement clock the bank does not control. A mistake there is a reversal, a support call and a potential duplicate debit.

The other retrieves documents and raises requests across eleven categories. A mistake there is a re-run. Designing both to the same standard means either over-engineering the second or under-engineering the first.

The organising axis

Classify by reversibility first. An irreversible action needs a state machine, idempotency and a designed reversal path before the happy path is signed off. A reversible one needs a clean taxonomy and per-category eligibility, and nothing heavier.

Everything else — identity, consent, documents, status, exceptions — is shared, and was.

THE NUMBERS THAT MATTER

1

axis that decides the design

6

layers where they diverge

5

components they shared

11

categories in the reversible journey

1

state machine, reused later

0

spine components forked

Why this matters at portfolio level: self-service programmes are usually planned by channel or by business area. Planning them by reversibility groups the journeys that genuinely share a design problem, and separates the ones that only look similar — which is what stops a payments-grade design being applied to a document download.

The journeys and their scope come from the delivery record; the comparison framework and the funnel figures are analysis

DELIVERED SCOPE Programme context The two journeys, their scope and the estate they landed in, as delivered during the 2024 engagement. - Two unassisted journeys - Eleven servicing request categories - External payment-rail integration - Shared platform and estate - Existing consent and document model - Enterprise status and reporting model	DELIVERY RECORD My ownership Journey design, data model and delivery to production for both journeys. - Reversible-transaction state machine - Servicing request taxonomy rebuild - Per-category eligibility rules - Idempotency and reversal design - Disposition and reporting continuity - Requirement through production	ILLUSTRATIVE Modelled Funnel, deflection and effort figures illustrate the argument. Directional, not measured outcomes. - Funding funnel figures - Per-category deflection spread - Design-effort comparison - Replace with bank actuals - Shape is the argument - Method transfers unchanged	OUT OF SCOPE Not claimed Explicitly excluded so the boundary is unambiguous. - Enterprise transformation outcomes - Any confidential screen or flow - Bank payment or servicing volumes - Independently audited results - Platform or architecture selection - Commercial terms of engagement
--	--	--	---

THE CONTRAST

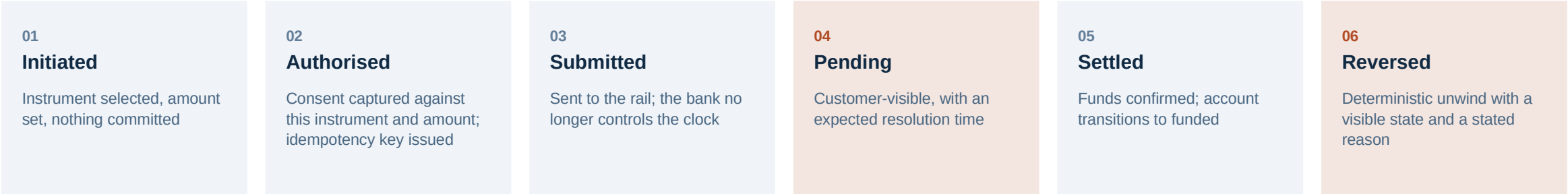
Same channel, same platform, same year — and almost nothing about the two designs is the same once you look past the interface

Design layer	Money movement — account funding	Information and requests — loan servicing
Failure cost	A wrong debit, a duplicate, or money in limbo	A re-run and a few wasted minutes
State model	Its own state machine with intermediate states	Simple request states on a shared model
Idempotency	Enforced at authorisation — a retry must not double-debit	Not required; repeat requests are harmless
Reversal	Designed and tested before the happy path	Not applicable
Timing	Bounded by an external settlement clock	Bounded only by the fulfilment system's own service level
Eligibility	One rule set — can this instrument fund this account	Eleven rule sets, one per request category
Hardest problem	The clock conflict between rail and session	The taxonomy, upstream of any interface work

What the table shows	Where the effort actually went	The planning implication
• Only the top rows of a requirements document would look alike. • Every row below that is a different problem with a different solution. • One self-service standard would be wrong for one of them.	• Funding: the state machine, reversal and idempotency — roughly two-thirds of the build. • Servicing: the taxonomy rebuild and eligibility rules — roughly two-thirds of the build. • In both cases almost none of it was the interface.	• Group self-service work by reversibility, not by channel or business area. • Reversible journeys can be batched and delivered quickly by one squad. • Irreversible ones need payments-grade attention however small.

Source: Author's design analysis

Funding is a state machine, not a field — and the reversal path was built and tested before the happy path was signed off



The two highlighted states are the ones a naive design omits — and they are the two that generate support calls.

Pending is a customer-facing state

- A rail that settles on its own timetable produces a window where the money has left and not arrived.
- If that window is invisible, the customer retries — and a retry without idempotency is a second debit.
- Naming the state, showing it, and giving it an expected resolution time removes most of the support volume.

Reversal is a first-class outcome

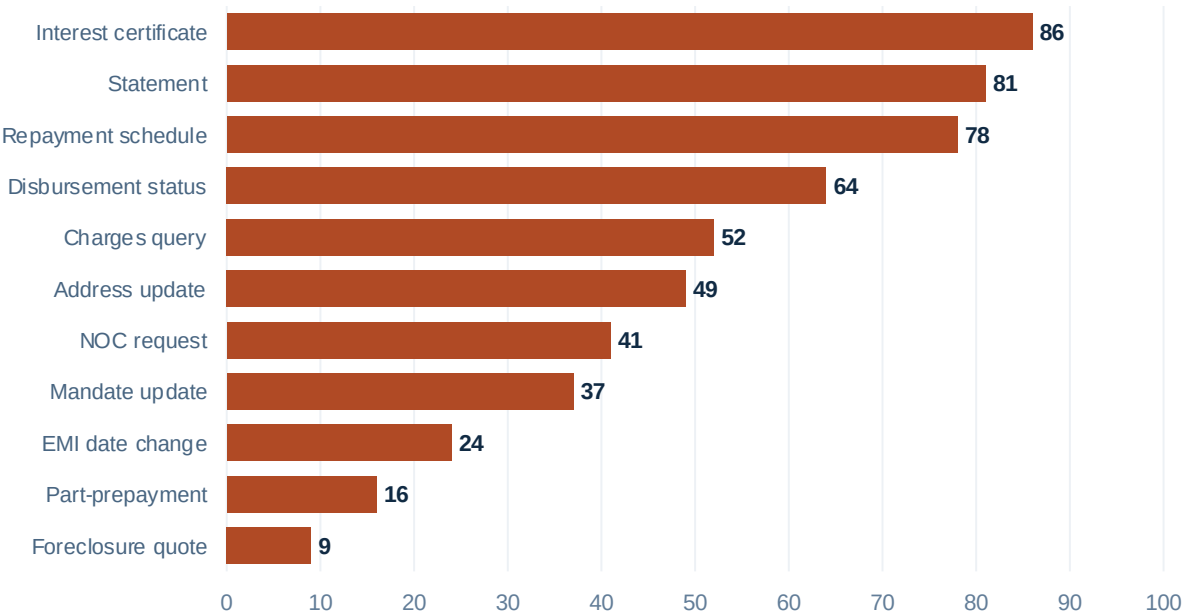
- Not an error path. A defined outcome with its own state, its own notification and its own reason code.
- Built and tested first, because a reversal discovered in production is discovered by a customer.
- It also has to be idempotent — a reversal retried must not credit twice.

Idempotency at authorisation

- The key is issued when consent is captured, not when the request is submitted.
- That way a network failure between the two cannot produce two authorised intents.
- This is the single detail that distinguishes a payments design from a form.

The state machine built here was inherited unchanged by the commercial card and group insurance journeys later in the year. That was not planned — it emerged because reversible-transaction handling turned out to be the shape of every journey that commits something on the customer's behalf. It is the clearest example in the portfolio of a component earning reuse by being built properly once.

Eleven categories deflect at wildly different rates — and the spread, not the average, is what should have set the phase-two roadmap



The pattern in the spread

- The top four are pure document retrieval — no judgement, no exception, no downstream owner. They approach full deflection.
- The middle band involves a change to a record, so they carry validation and occasionally a review.
- The bottom three move money or end a contract. They barely deflect, and forcing them to would have been a mistake.
- The ordering is almost exactly reversibility again — the same axis that separated the two journeys separates the categories inside one of them.

Deflection by category, calibrated to published financial-services benchmarks

Why the taxonomy work came first

The eleven categories did not arrive as eleven. They arrived as an accreted set built around back-end system boundaries, with overlapping definitions and no consistent notion of what completion meant.

Rebuilding the taxonomy from customer language, then mapping back to owning systems, was roughly two-thirds of the effort and none of the demonstration. It is also the only reason the deflection spread is interpretable at all — a spread measured across badly defined categories tells you nothing about what to do next.

WHAT THEY SHARED

Five components carried both journeys unchanged — the divergence was entirely in the layer above them

Identity and entitlement	Consent	Document and evidence	State and service level	Exception and escalation
Who the customer is and what they hold. Identical requirement in both.	Purpose-bound and timestamped. The funding journey captures consent per instrument; the servicing journey per request. Same mechanism.	One store, one retention policy. Servicing retrieves from it; funding writes authorisation evidence to it.	A common status vocabulary, so both journeys stay visible to enterprise reporting without inventing private states.	One queue, one route back to a human. Used heavily by funding reversals and by the low-deflection servicing categories.

The ratio that matters

- Roughly eighty per cent of what each journey needed was already in production before either started.
- The twenty per cent that was specific is where all the design difficulty sat — and it was completely different in each case.
- That ratio is the argument for building the spine before the second journey, not the fifth.

What would have happened otherwise

- Two consent implementations, two document stores, two status vocabularies and two exception queues.
- Enterprise reporting would have needed a translation layer within a quarter.
- And the second journey would have cost roughly what the first did, rather than substantially less.

The discipline this needed

- Neither journey was allowed to fork a shared component, including when a fork would have been faster than sprint.
- The servicing journey did extend the exception queue with per-category routing, which is the right kind of change.
- Extending is healthy. Forking is the failure.

Four judgements for designing self-service at portfolio scale

- | | | |
|----|--|--|
| 01 | Classify by reversibility, not by channel | Two journeys in the same channel can need completely different design standards. Reversibility predicts which one needs a state machine, idempotency and a designed reversal — and which needs none of it. |
| 02 | Design the reversal before the happy path | For anything irreversible, the unwind is a first-class outcome with its own state, notification and reason code. A reversal path discovered in production is discovered by a customer. |
| 03 | Fix the taxonomy before the interface | Where a journey spans many request types, the category definitions are the work. A deflection rate measured across badly defined categories cannot tell you what to build next. |
| 04 | Read the spread, never the average | A blended deflection figure hides both the categories that deflect completely and the ones that never will. The spread is what sets the next phase; the average is what gets reported. |

Both journeys removed work from a human. Only one of them could take money from a customer and fail to give it back — and every meaningful design difference between them follows from that single fact.